



Christine Chau



RÉCAPITULATIF PROFESSIONNEL

Suite à une carrière de 15 années dans le secteur de la mode et du Luxe, pour des marques internationales, en tant que Visual Merchandiser, J'ai souhaité me reconverter dans le Développement Web en lien avec le E-Merchandising. Mes intentions de départ, étaient animées par une extension de compétences et ont rapidement évoluées en passion et reconversion, pour le développement Back-End. Mon désir de développer des projets web innovants et créatifs m'ont poussé à me reconverter dans le développement Web spécialisé en PHP SYMFONY.

Diplômes

2007 **Décoratrice, étalagiste, Merchandiser Homologué Niveau III (BTS)**
spécialisé dans l'agencement intérieur des espaces commerciaux et événementiels.

2002 **DMA diplôme des métiers d'arts Homologué Niveau III (BTS),**
spécialisation arts appliqués aux industries de l'ameublement
Ecole BOULLE

Expériences Professionnelles

. Senior Visuel Merchandiser Coordinator
RALPH LAUREN I
Septembre 2012 - Mars 2022.(10 ans)

.Windows Dresser /Visuel Merchandiser
DSQUARED2 2011 (1 an)

Présentation Projet

<https://quai-antique.christine-chau-projets.com>



AMBIANCE



#D70000

Guardsman Red

#FFFFFF

White

#000000

Black



Présentation Projet

Ce projet concerne une application Web vitrine, pour un restaurant gastronomique le Quai Antique, situé en Savoie à Annecy, sous la direction du chef Arnaud Michant.

Cette plateforme permet aux visiteurs d'accéder à toutes les informations relatives au restaurant et aux clients, inscrits, de réserver une table. La direction du Restaurant (hôte d'accueil et son équipe) peut gérer toutes les réservations, inscrire un client, modifier les horaires du restaurant, modifier la carte et ses menus.

Suivant la volonté du Quai Antique, les places disponibles sont affichées lors de la réservation par date et heure, les allergies alimentaires des clients inscrits, sont remplis automatiquement lors de chaque réservation, le personnel du Quai Antique peut modifier la galerie d'images sur la page d'accueil et les horaires sont affichés, sur chaque pied de page, lors de la navigation.

J'ai réalisé ce projet seule sur une durée de 4 semaines.

J'ai choisie de travailler avec Adobe XD pour le maquettage et PHPStorm, comme environnement de développement intégré.

projet visible : <https://quai-antique.christine-chau-projets.com>

Gestion Projet Global

The screenshot shows a Trello project board titled "PROJET ECF QUAI ANTIQUE CHRISTINE CHAU". The board is organized into five columns: "FRONT-END", "BACK-END", "à faire", "En cours", and "Terminer". Each column contains a list of tasks represented by cards. The "FRONT-END" column has five cards, including "6. Réalisation complète des pages du site_avec Bootstrap / Responsive" and "Réalisation README.MD". The "BACK-END" column has five cards, including "1. Réalisation du diagramme de classe" and "5. Ajout de Fixtures dans la BDD". The "à faire" column has four cards, including "1. Recherches et définition Logo et images" and "5. Réalisation de la Barre de Navigation". The "En cours" and "Terminer" columns are currently empty, each with a "+ Ajouter une carte" button. The interface includes a top navigation bar with "Espaces de travail", "Récemment", "Favoris", "Modèles", and a "Créer" button. A search bar labeled "Parcourir" is also present. The board is set to "Public" and has a "Tableau" view selected.

PROJET ECF QUAI ANTIQUE CHRISTINE CHAU ☆ Public Tableau

Power-ups ⚡ Automatisation ⚙ Filtre CC Partager ...

FRONT-END

- 6. Réalisation complète des pages du site_avec Bootstrap / Responsive
- 8. création d'un fichier pour l'affichage des réservations par jour et par heure
- 10. Création d'un fichier Twig pour modifier galerie images / ADMIN
- 11. Création d'un fichier twig pour mon espace Client/ admin
- Réalisation README.MD
- Réalisation documentation Technique

BACK-END

- 1. Réalisation du diagramme de classe, utilisation et de séquence
- 2. Création et définition des Routes avec les Controllers
- 3. Créer et administrer une base de données avec Doctrine/symfony
- 4. Création User et gestion des rôles avec authentification par Email
- 5. Ajout de Fixtures dans la BDD avec Doctrine

à faire

- 1. Recherches et définition Logo et images
- 2. Réalisation Planche graphique et Planche d'ambiance
- 3. Maquetter l'application avec ADOBE XD
- 4. Réalisation de la page d'accueil Twig + Bootstrap
- 5. Réalisation de la Barre de Navigation

En cours

+ Ajouter une carte

Terminer

+ Ajouter une carte

+ Ajouter une carte

Gestion Projet Global

"Pour la gestion du projet "Le Quai Antique", j'ai utilisé l'outil de gestion de projet Trello. Trello est un système de tableau Kanban numérique qui m' a permis d'organiser et de suivre l'avancement du projet de manière visuelle et intuitive.

La méthode Kanban est un moyen efficace de gérer le travail qui met l'accent sur la visualisation du travail, le maintien d'un flux constant de tâches achevées, et l'amélioration continue du processus.

Dans Trello, j'ai créé différents tableaux pour représenter les diverses phases du projet, allant de la conception à la réalisation. Ces tableaux étaient divisés en listes, chacune représentant une étape différente du processus du projet. Deux catégories : Front-end, Back-end, avec 3 statuts: A faire, En cours, Terminé.

Chaque tâche du projet était représentée par une carte Trello. Ces cartes contenaient des détails sur la tâche, y compris sa description, sa date d'échéance, les membres de l'équipe assignés à la tâche, et tout autre commentaire ou pièce jointe pertinent.

Gestion Projet Conception Front-End



- à faire

1. Recherches et définition Logo et images
2. Réalisation Planche graphique et Planche d'ambiance
3. Maquetter l'application avec ADOBE XD
4. Réalisation de la page d'accueil Twig + Bootstrap
5. Réalisation de la Barre de Navigation
- + Ajouter une carte



Police Lora

Titre

Medium 500

Whereas recognition of the inherent digni

Texte

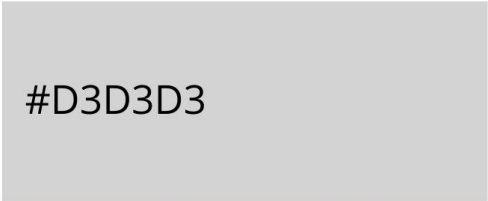
Regular 400

Whereas recognition of the inherent digni



#D700000

#FFFFFF



#D3D3D3



#02080C

Gestion Projet Maquettes / page accueil

à faire

1. Recherches et définition Logo et images



2. Réalisation Planche graphique et Planche d'ambiance



3. Maquetter l'application avec ADOBE XD



4. Réalisation de la page d'accueil Twig + Bootstrap



5. Réalisation de la Barre de Navigation

+ Ajouter une carte



iPhone X, XS, 11 Pro – 1



iPhone X, XS, 11 Pro – 2



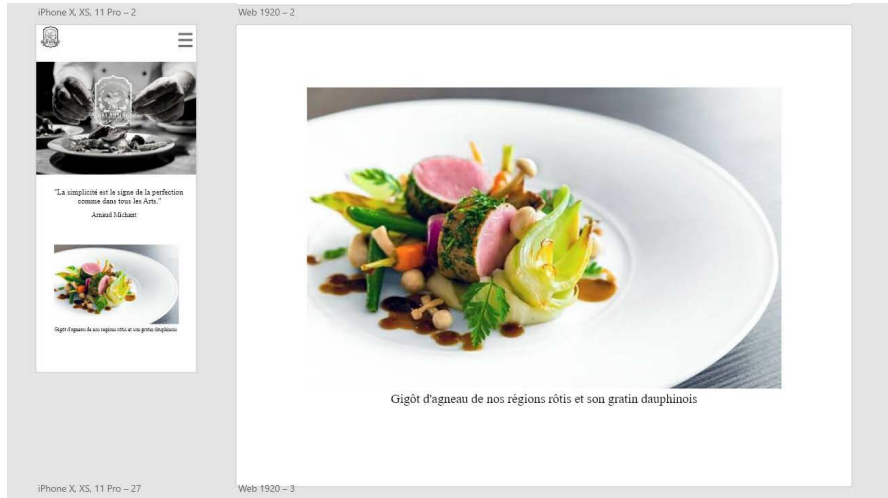
Web 1920 – 1



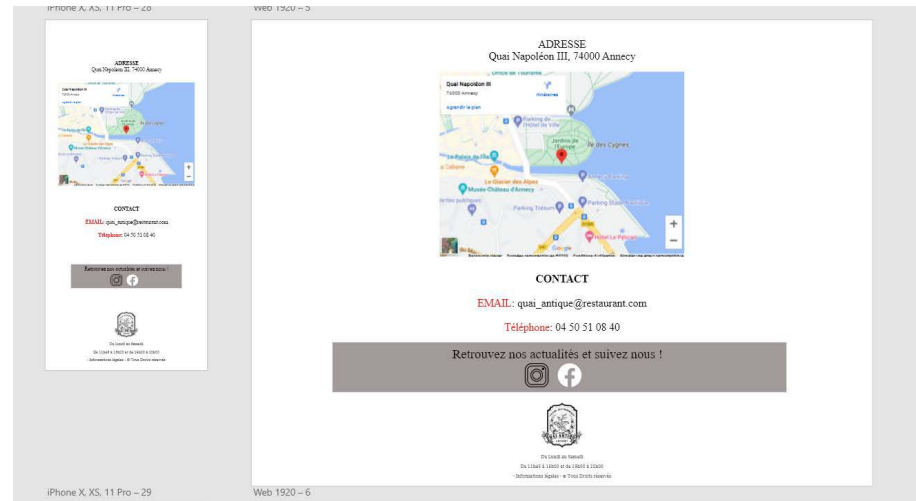
Web 1920 – 2



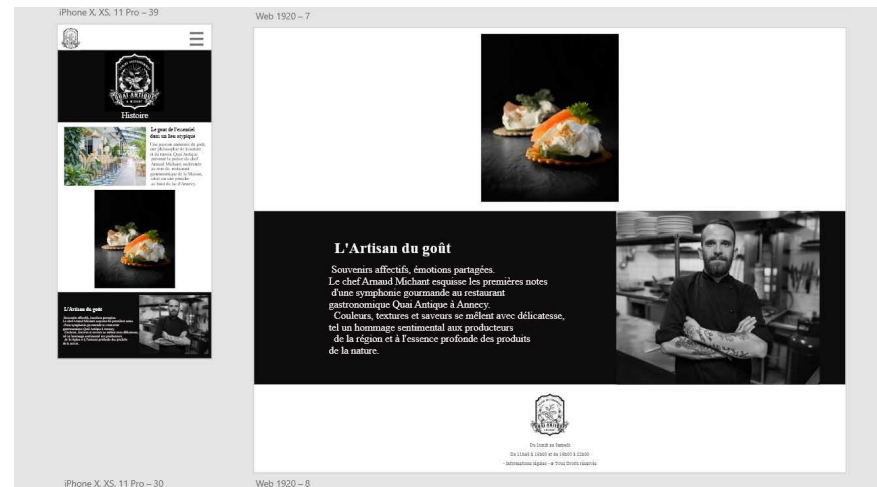
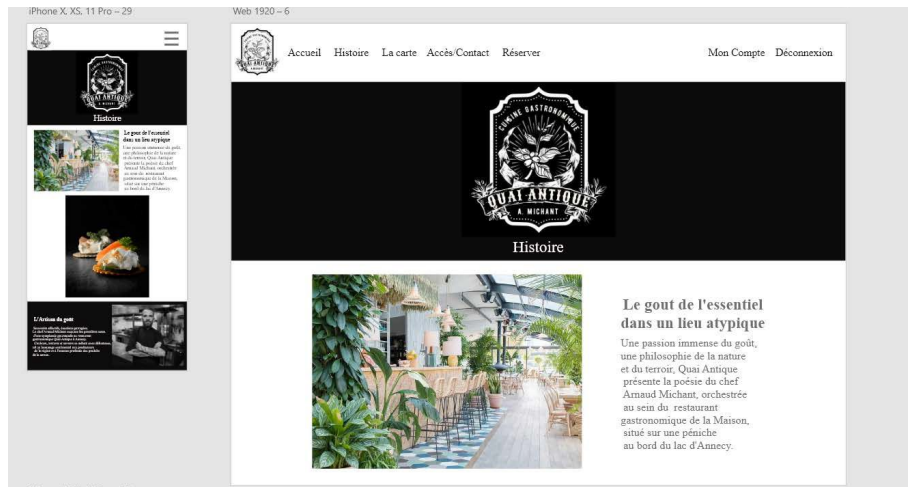
Maquettes Conception / page accueil



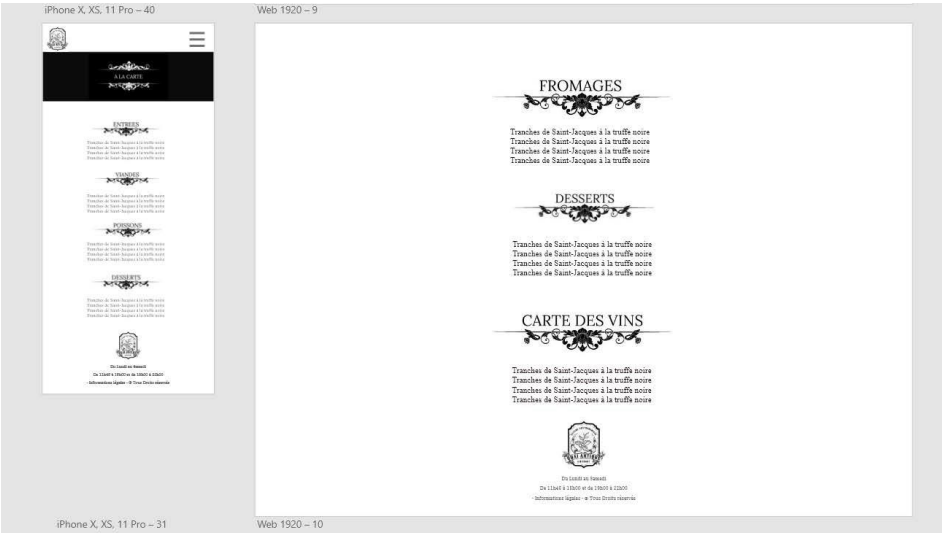
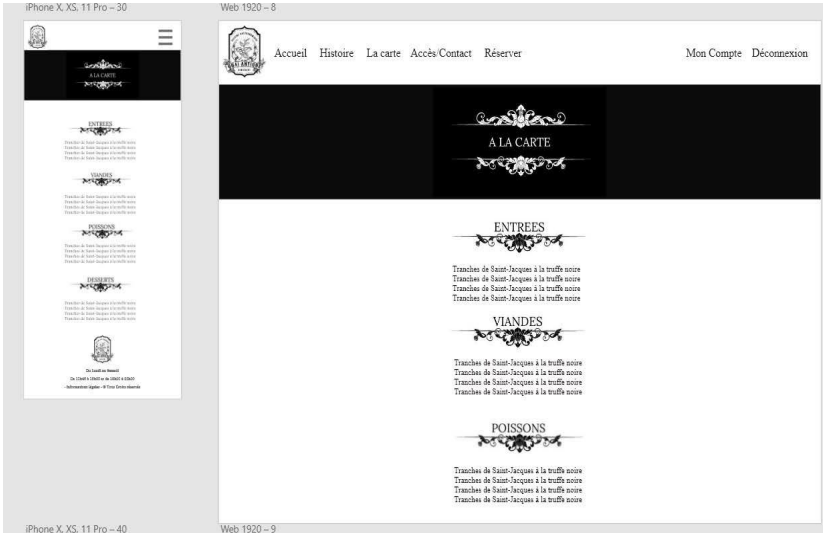
Maquettes Conception / page accueil



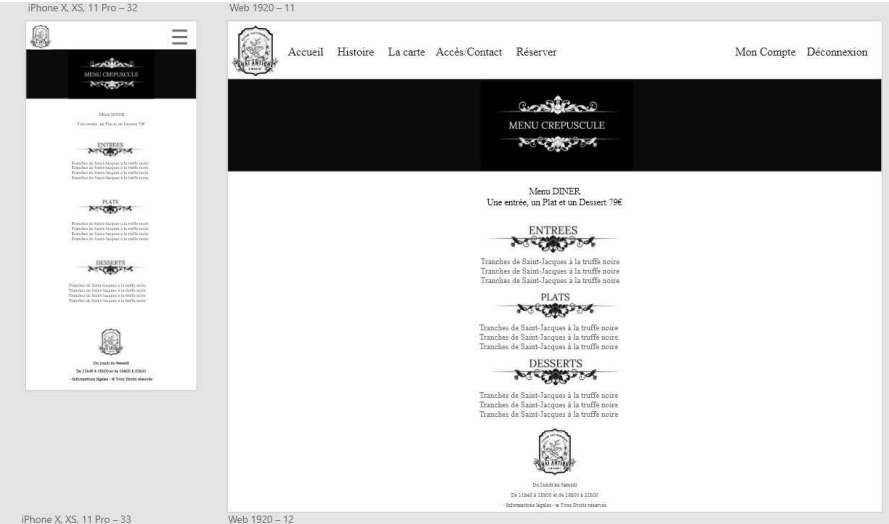
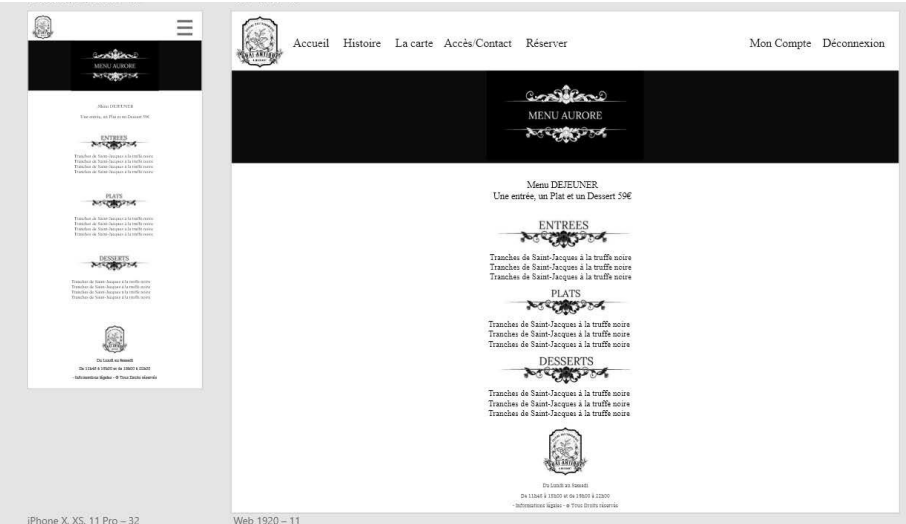
Maquettes Conception / page Histoire



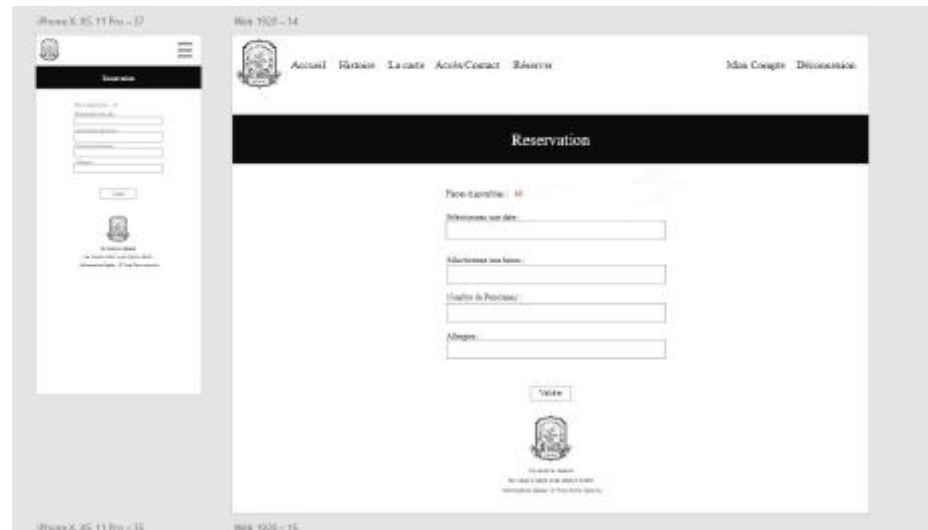
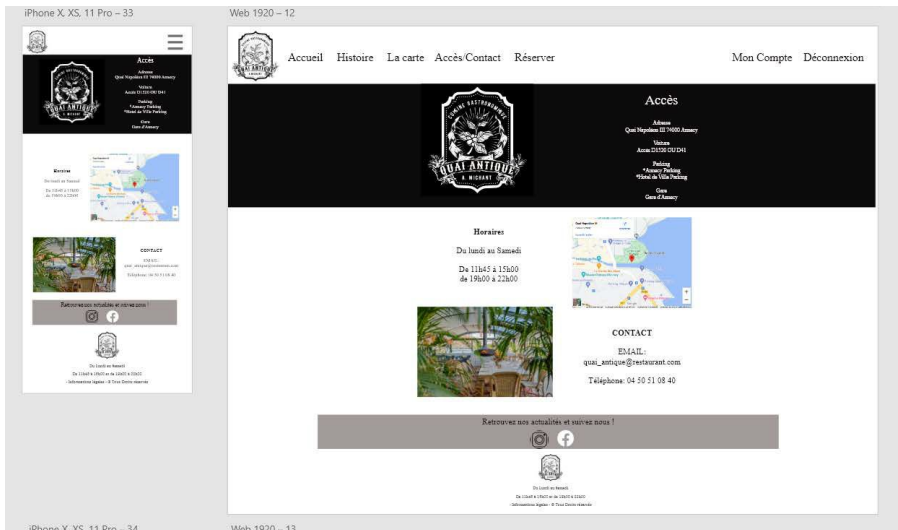
Maquettes Conception / page A la Carte



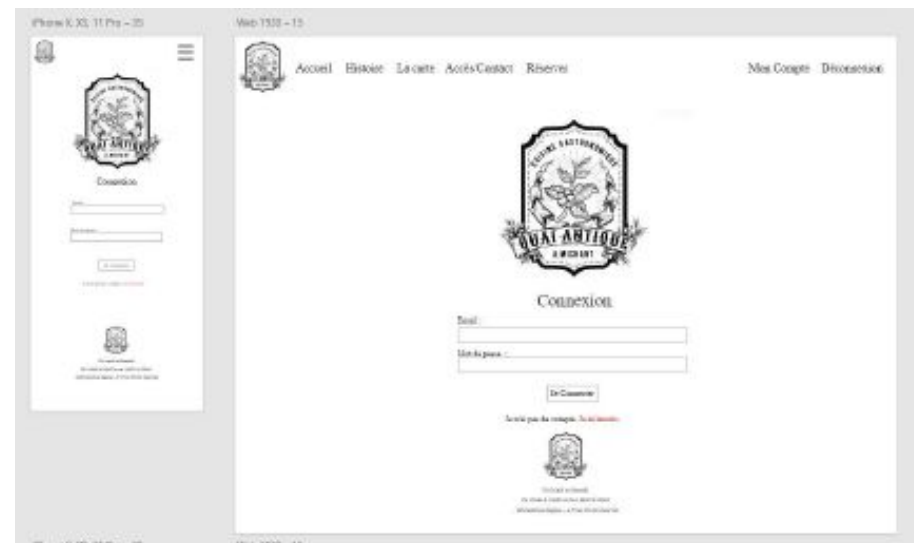
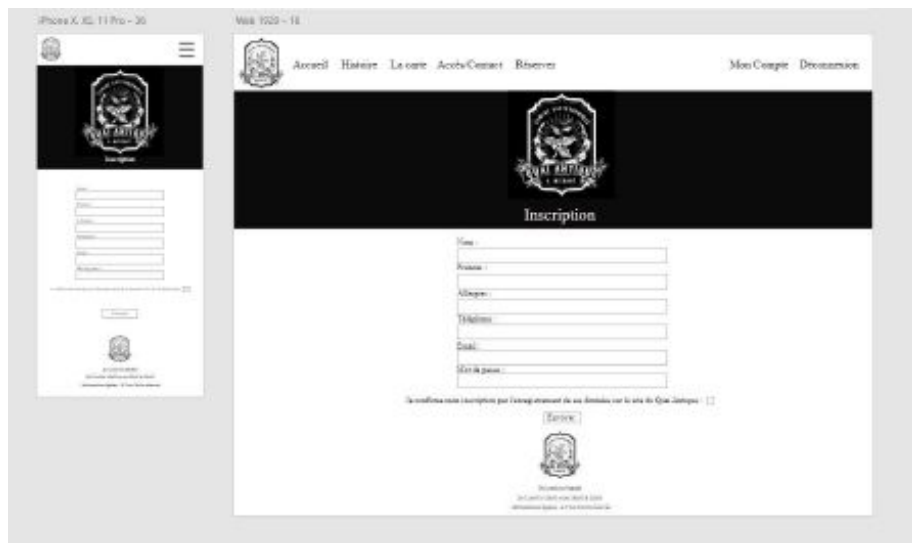
Maquettes Conception / page Menu Déjeuner & Menu Dîner



Maquettes Conception / page Accès-Contact & Réservations



Maquettes Conception / page Inscription & Connexion



Environnements Techniques

FRONT-END



BACK-END



ENVIRONNEMENT



PhpStorm

phpMyAdmin



Gestion Projet Conception Back-end

BACK-END

1. Réalisation du diagramme de classe, utilisation et de séquence

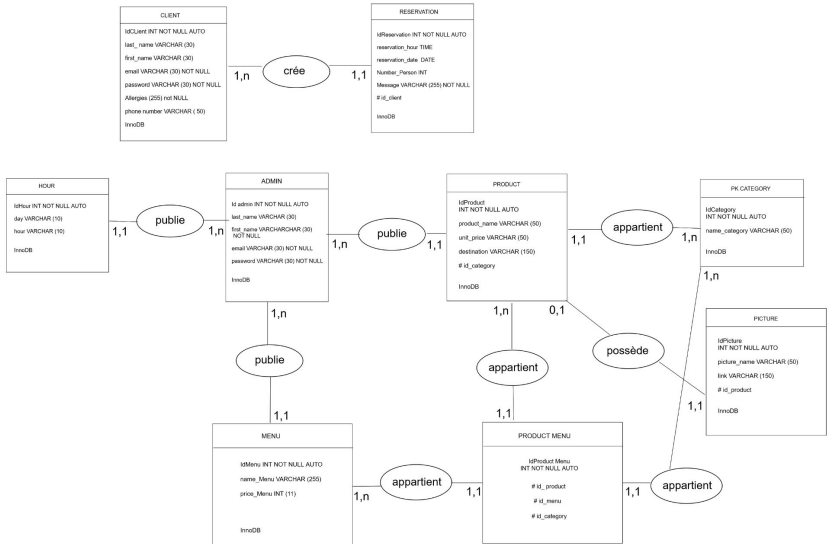
2. Création et définition des Routes avec les Controllers

3. Créer et administrer une base de données avec Doctrine/symfony

4. Création User et gestion des rôles avec authentification par Email

5. Ajout de Fixtures dans la BDD avec Doctrine

Schéma MCD _QUAI ANTIQUE



Outils de modélisation de données

1. **Schéma MCD (Modèle Conceptuel de Données) avec la méthode Merise** : "Pour la conception de la base de données, j'ai utilisé la méthode Merise pour créer un Modèle Conceptuel de Données (MCD). Le MCD m'a permis d'identifier et de définir les entités importantes du système, comme le client, la réservation, la table, etc. De plus, il a permis de déterminer les relations entre ces entités, par exemple, un client peut faire plusieurs réservations, mais une réservation ne peut concerner qu'un seul client. Cette approche a fourni une représentation claire et structurée de la base de données, facilitant ainsi son développement ultérieur."
2. **Diagramme d'utilisation (ou diagramme de cas d'utilisation) pour un utilisateur et un administrateur** : "Pour définir les interactions entre les utilisateurs du système et les fonctionnalités, j'ai créé des diagrammes de cas d'utilisation pour un utilisateur (User) et un administrateur (Admin). Chaque diagramme de cas d'utilisation représente les différentes actions que chaque type d'utilisateur peut effectuer dans le système. Par exemple, un User peut se connecter, faire une réservation, etc., tandis qu'un Admin peut ajouter/modifier les informations du restaurant, gérer les réservations, etc. Ces diagrammes ont aidé à clarifier les rôles, les responsabilités de chaque type d'utilisateur et les fonctions qui leur sont attribuées."
3. **Diagramme de séquence pour la fonction "réserver une table"** : "Enfin, pour comprendre le flux d'interactions pour la fonction "réserver une table", j'ai créé un diagramme de séquence. Ce diagramme illustre comment les différents objets dans notre système interagissent sur une période de temps pour réaliser la fonction de réservation d'une table. Par exemple, il commence par l'utilisateur qui effectue une demande de réservation, puis le système qui vérifie la disponibilité de la table, et enfin confirme ou refuse la réservation à l'utilisateur. Ce diagramme de séquence m'a aidé à comprendre et à définir le comportement du système de réservation."

Diagramme de cas d'utilisation User & Admin

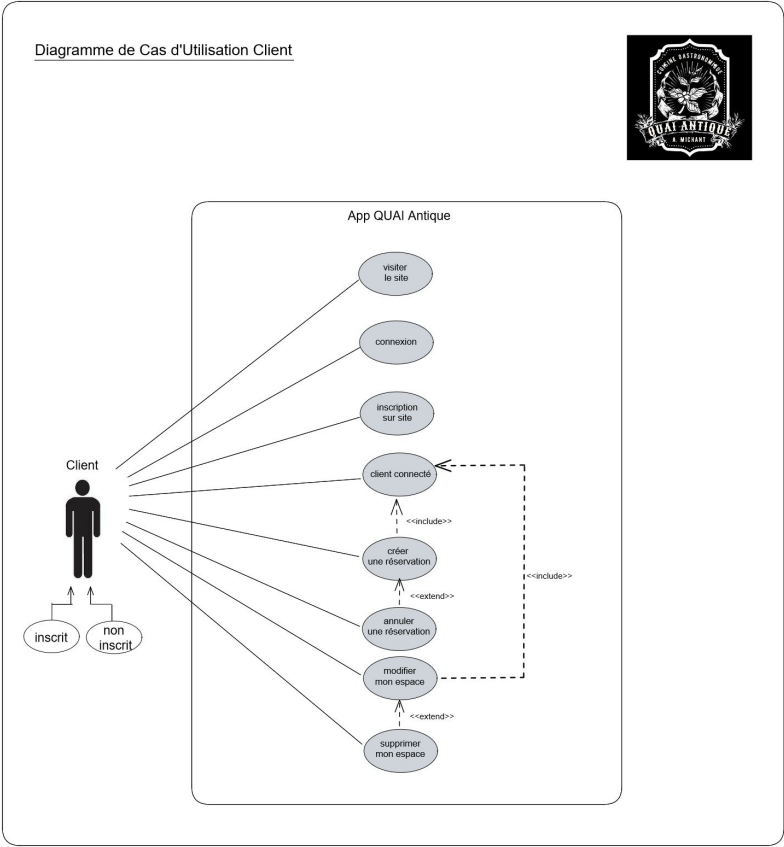
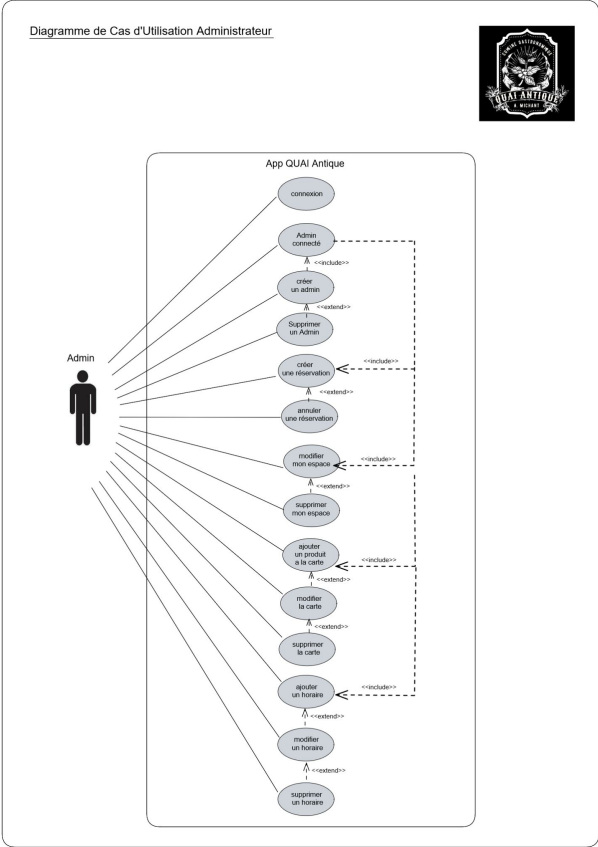
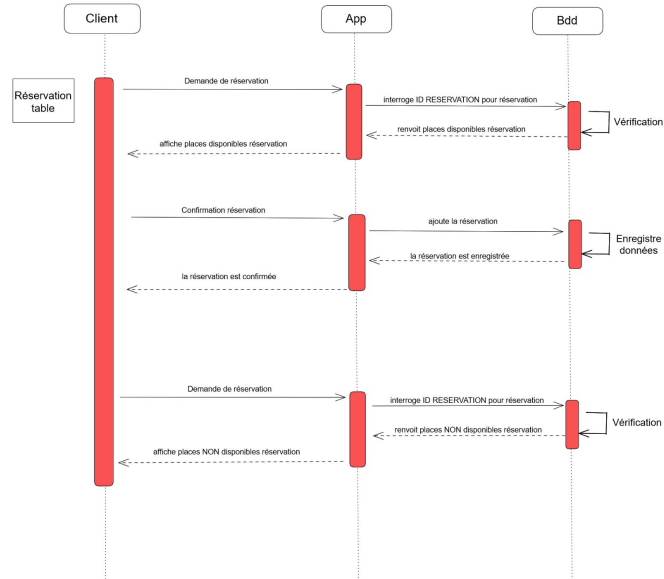


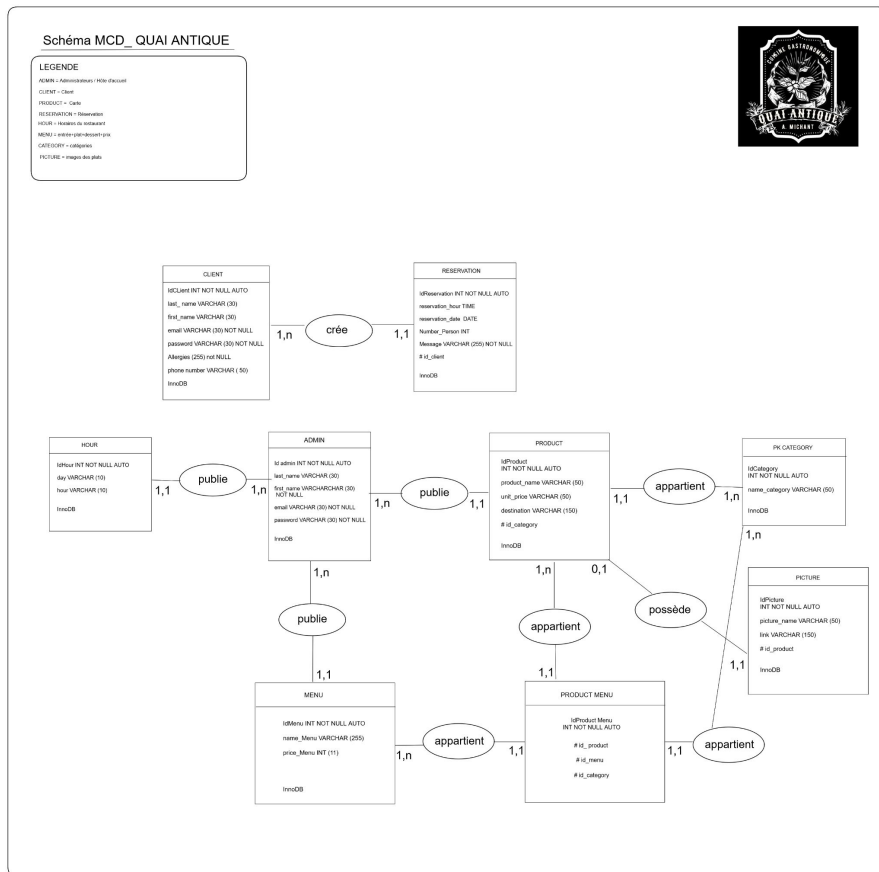
Diagramme de séquence US6 Réserver une Table

Diagramme de séquence QUAI ANTIQUE

CLIENT: "USA6 - Réserver une table"



Connexion User



Jeu de données

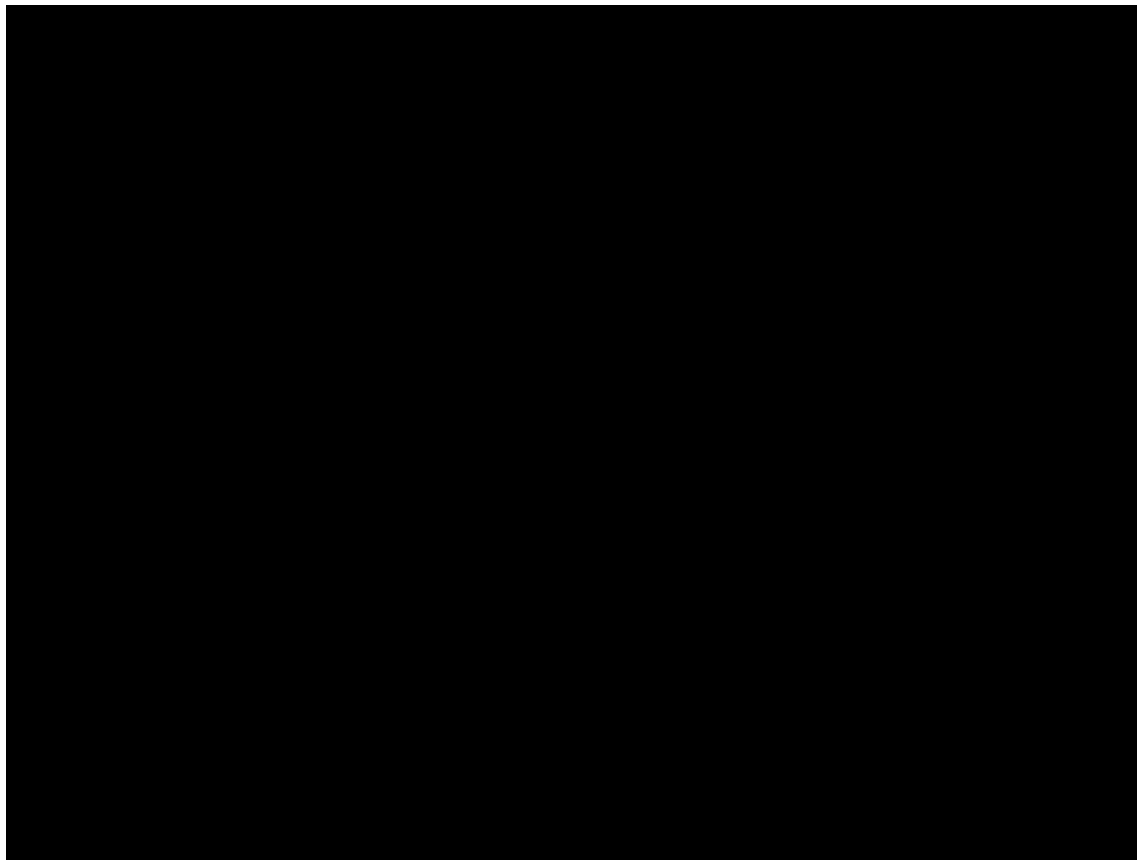
User: marine@gmail.com
password: fevrier

Fonctionnalités attendues:

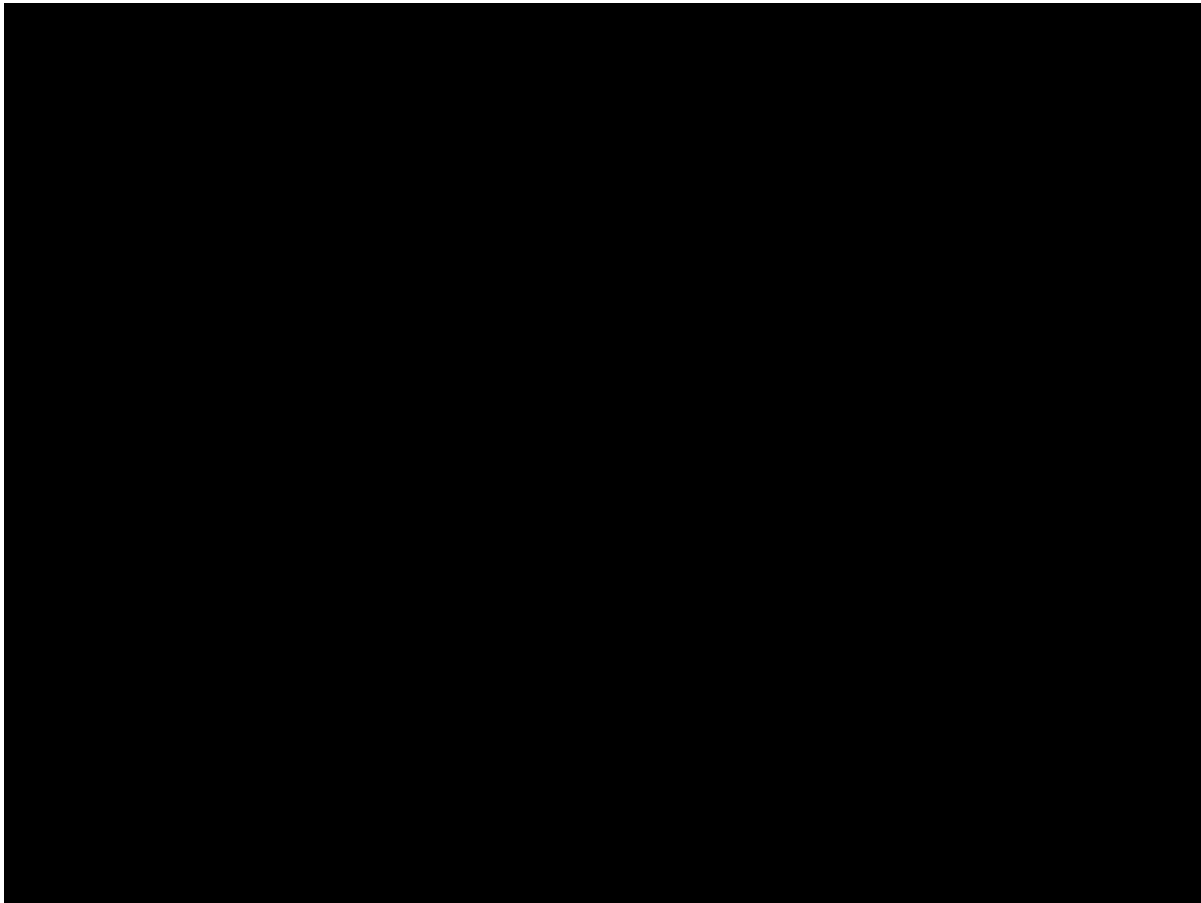
- connexion à “Mon compte”
- réserver une table

<https://quai-antique.christine-chau-projets>

Connexion User & Réserver une table



Connexion User/ solution



Connexion User/ solution

"Dans le cadre de ce projet, j'ai mis en place une fonction d'authentification pour nos utilisateurs. L'authentification est cruciale pour la sécurité de nos utilisateurs et de nos données. Laissez-moi vous expliquer comment cela a été réalisé.

D'abord, j'ai utilisé l'entité `User`. Celle-ci stocke les informations essentielles de chaque utilisateur, notamment l'e-mail, les rôles, le nom, le prénom, les allergies et le numéro de téléphone. Un point crucial ici est la façon dont nous stockons le mot de passe. Au lieu de conserver le mot de passe en texte clair, nous enregistrons une version hachée. C'est une pratique de sécurité standard qui rend le mot de passe illisible, même si quelqu'un parvenait à accéder à nos données.

Ensuite, pour générer notre système d'authentification, j'ai utilisé la commande `make:auth` de Symfony, qui a créé automatiquement une classe `AppLoginAuthenticator`. Cette classe contient toute la logique d'authentification, y compris la validation des identifiants de l'utilisateur et la gestion des échecs d'authentification.

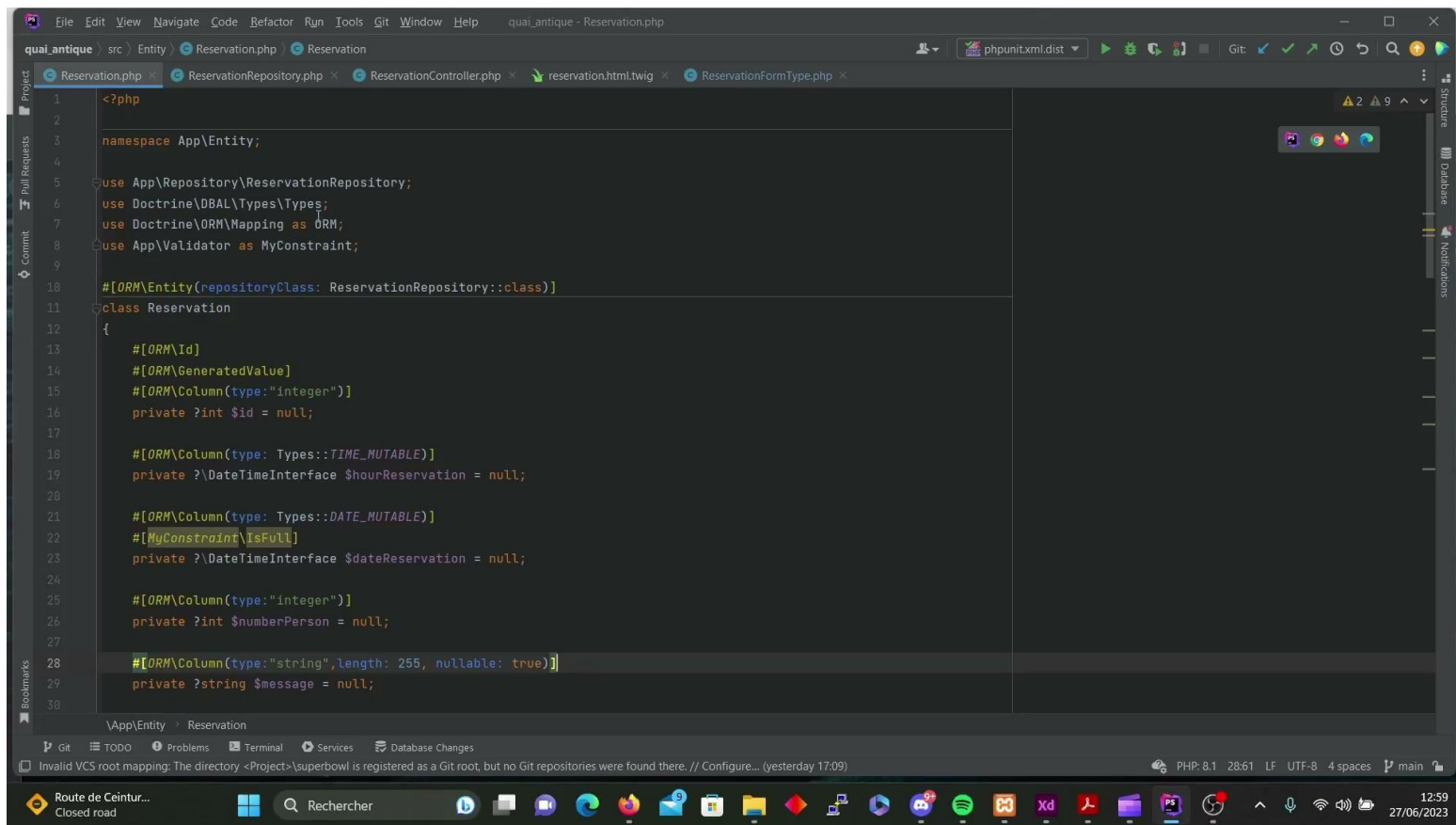
Un élément de sécurité essentiel dans ce processus est le badge `CsrfTokenBadge`. CSRF, qui signifie Cross-Site Request Forgery, est une attaque qui trompe la victime pour qu'elle effectue une action non désirée. Pour nous prémunir contre cela, j'utilise un token CSRF, qui est un jeton unique associé à la session de l'utilisateur. Ce jeton est vérifié à chaque demande pour s'assurer qu'il est valide et correspond à la session de l'utilisateur.

Enfin, j'ai créé un `SecurityController` pour gérer les requêtes de connexion et de déconnexion. Lorsqu'un utilisateur tente de se connecter, nous utilisons `AuthenticationUtils` pour récupérer les informations de la dernière authentification et les erreurs éventuelles. Pour la déconnexion, nous n'avons pas besoin de mettre en œuvre une logique spécifique car Symfony gère cela pour nous.

Le fichier `login.html.twig` est le template pour la page de connexion. Ici, nous nous protégeons contre les attaques XSS, qui consistent à injecter du code malveillant dans la page, en utilisant le système de templating Twig, qui échappe automatiquement les variables insérées dans la page.

En résumé, nous avons mis en place un système d'authentification robuste qui protège contre plusieurs types d'attaques courantes tout en offrant une expérience utilisateur fluide. Les principes de sécurité intégrés dans notre code sont le hachage des mots de passe, la protection contre les attaques CSRF grâce à l'utilisation de jetons uniques, et la protection contre les attaques XSS grâce à l'échappement automatique des variables avec Twig."

Réserver une Table / Solution



Je vais vous présenter comment j'ai implémenté la fonctionnalité de réservation d'une table dans un restaurant en utilisant Symfony, un framework PHP. Cette fonctionnalité est composée de plusieurs composants, notamment une entité "Réservation", un formulaire, un contrôleur et des fichiers de vue Twig pour la présentation.

D'abord, j'ai commencé par définir l'entité "Réservation" qui représente la table de réservation dans la base de données. Cette entité contient plusieurs propriétés, y compris l'heure et la date de la réservation, le nombre de personnes et un éventuel message de l'utilisateur. Cette entité est associée à l'entité "User" par une relation ManyToOne, ce qui signifie qu'un utilisateur peut avoir plusieurs réservations.

Ensuite, j'ai créé un "Repository" pour l'entité "Réservation", qui contient les méthodes pour interagir avec la base de données. Par exemple, j'ai créé une méthode `findNumberPerson` qui renvoie le nombre total de personnes qui ont réservé à une certaine date et heure. Cela permet d'assurer que nous n'acceptons pas plus de réservations que le restaurant ne peut en gérer.

Après cela, j'ai mis en place le contrôleur "ReservationController". Ce contrôleur contient plusieurs méthodes, notamment l'index qui affiche le formulaire de réservation, et une méthode `checkAvailability` qui vérifie la disponibilité d'une table en appelant la méthode `findNumberPerson` du Repository.

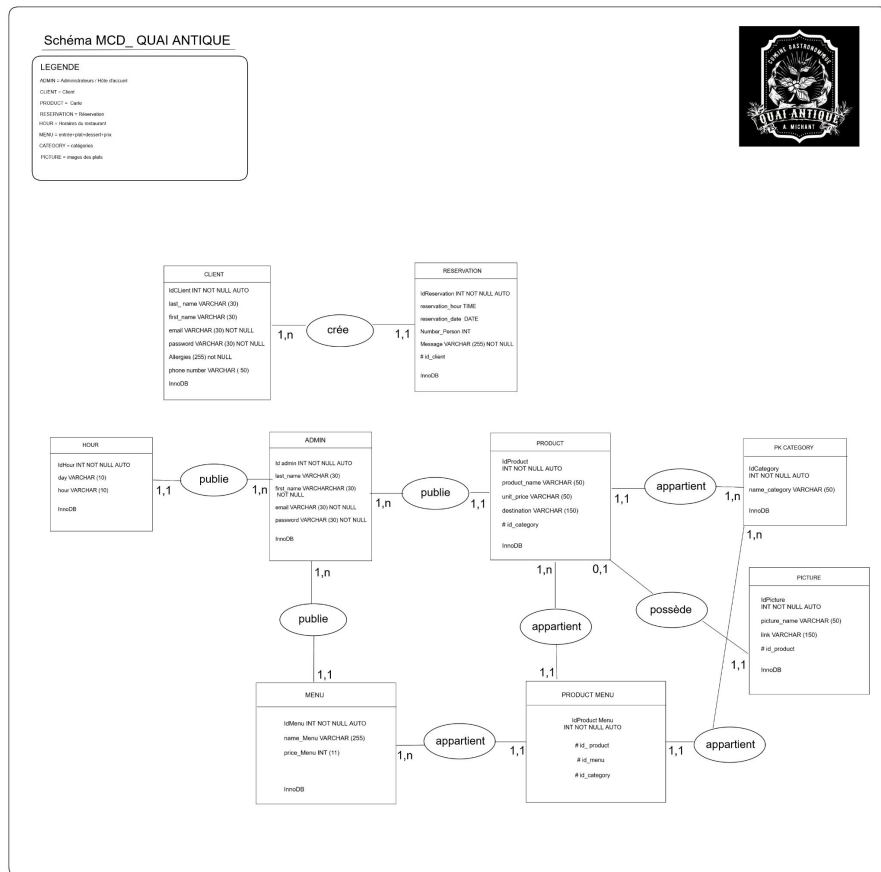
Le formulaire de réservation est défini dans "ReservationFormType". C'est ici que je spécifie quels champs sont inclus dans le formulaire et comment ils doivent être rendus. Par exemple, le champ de la date de réservation est un champ de type 'Date' avec un widget de sélection de date.

Enfin, la vue "reservation.html.twig" est le fichier où le HTML est écrit pour afficher la page de réservation. Dans ce fichier, j'ai inclus un bloc JavaScript pour rendre le formulaire interactif. Par exemple, chaque fois que l'utilisateur change la date ou l'heure de la réservation, une requête AJAX est envoyée pour vérifier la disponibilité de la table.

AJAX signifie Asynchronous JavaScript and XML. C'est une technique de développement web qui permet à une page web de communiquer avec un serveur, récupérer des données et mettre à jour la page de manière asynchrone, sans avoir à recharger l'ensemble de la page.

En conclusion, le processus de réservation commence lorsque l'utilisateur arrive sur la page de réservation, remplit le formulaire et soumet la réservation. Le contrôleur reçoit ensuite cette demande, vérifie la disponibilité de la table, puis sauvegarde la réservation si tout est en ordre. Sinon, un message d'erreur est renvoyé à l'utilisateur. En fin de compte, le système de réservation assure une gestion efficace de l'occupation des tables du restaurant tout en fournissant une expérience utilisateur fluide et interactive.

Admin / changer un plat à la carte



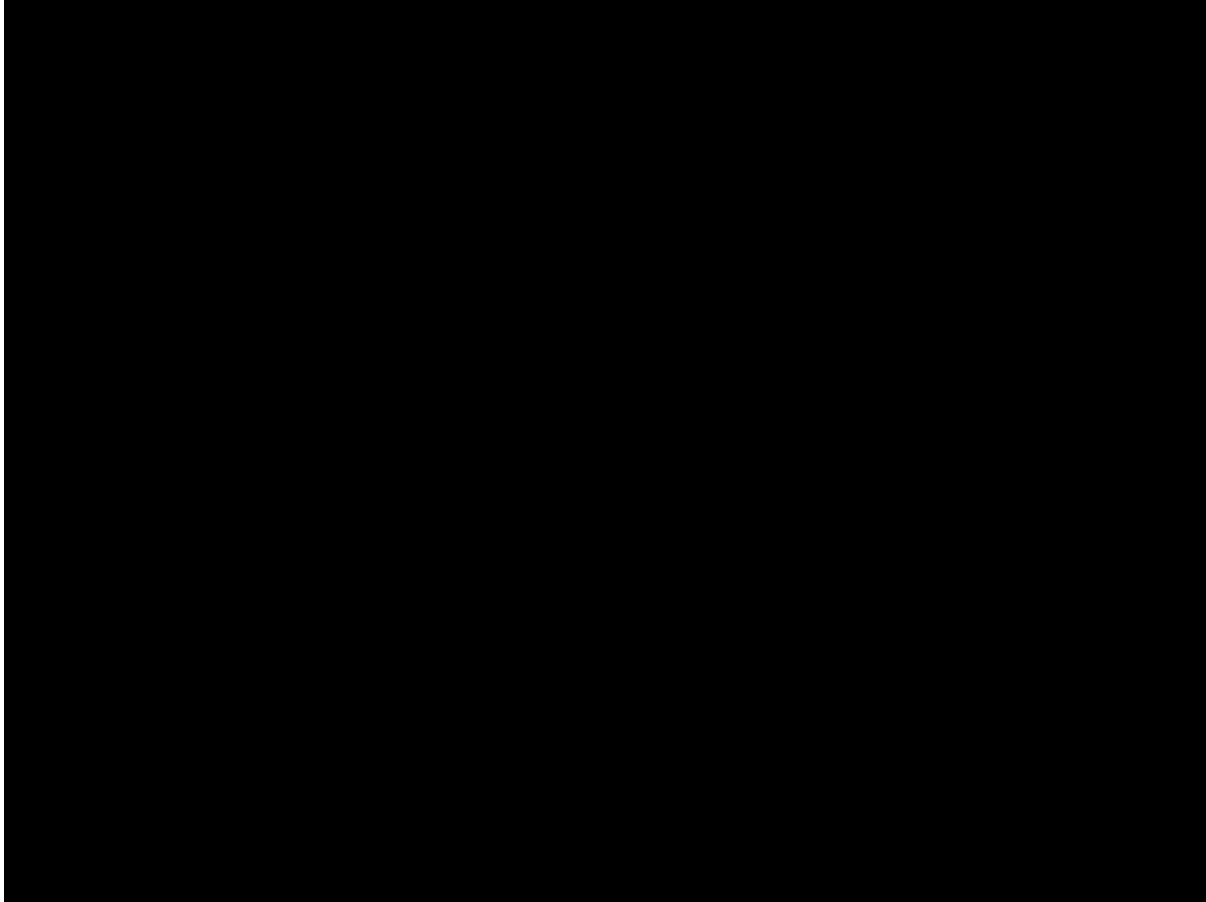
Jeu de données

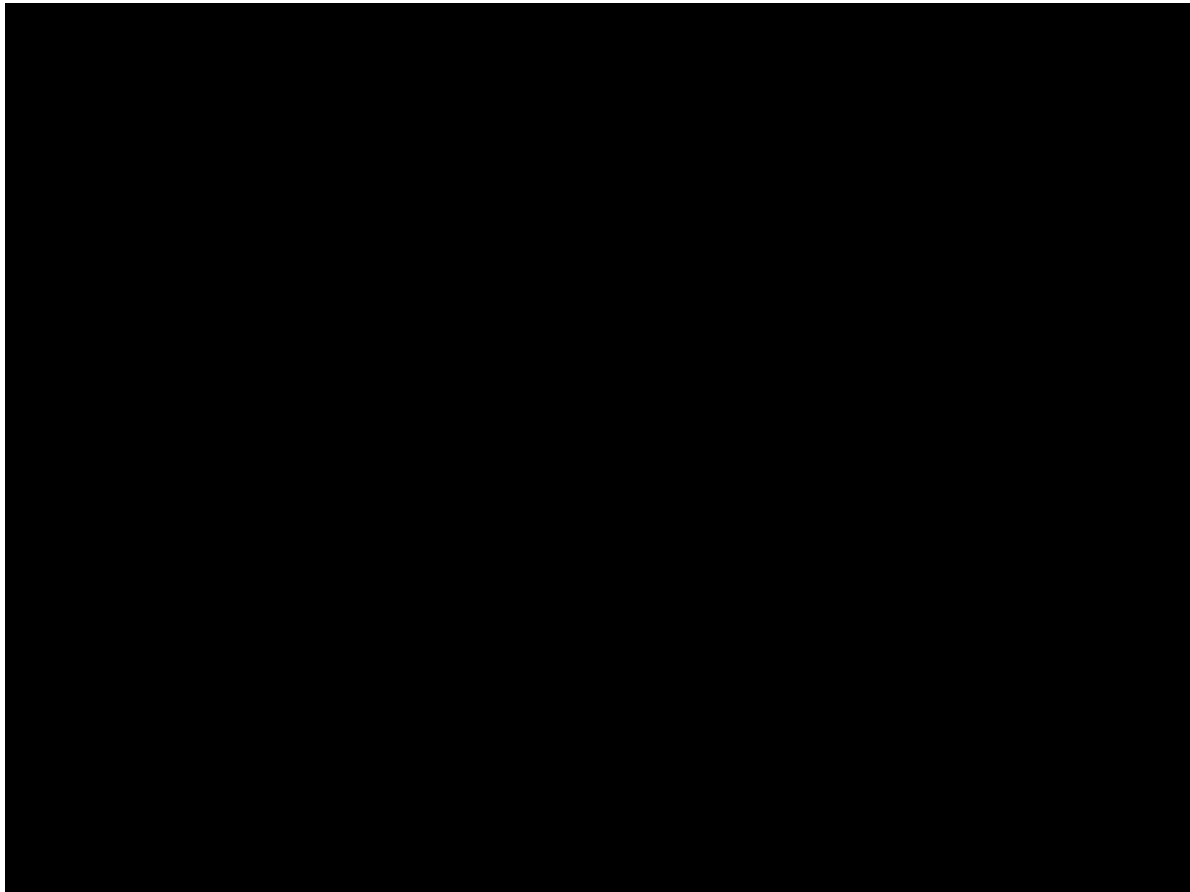
User: martin@gmail.com
password: fevrier

Fonctionnalités attendues:

- connexion à "Mon espace Admin"
- Modifier un plat à la carte

<https://quai-antique.christine-chau-projets>





Cette application a été construite pour optimiser l'administration du restaurant, en permettant à l'administrateur de gérer facilement différents aspects du restaurant, tels que la mise à jour des produits de la carte.

Dès qu'un administrateur se connecte sur le portail d'administration, il est authentifié et reconnu en tant qu'administrateur, puis dirigé vers l'index des administrateurs. Pour la sécurité de l'application, le processus d'authentification est très important. La méthode de hachage des mots de passe est utilisée pour assurer la sécurité des données de l'administrateur, garantissant ainsi que même en cas de fuite de données, les informations sensibles de l'administrateur ne sont pas exposées.

La page d'accueil de l'administrateur donne une vue d'ensemble de tous les administrateurs du site. De là, l'administrateur peut ajouter de nouveaux administrateurs, modifier les informations des administrateurs existants ou même supprimer un administrateur. Toutes ces actions sont gérées par le contrôleur AdminController qui utilise le repository UserRepository pour interagir avec la base de données.

Pour modifier un produit à la carte, l'administrateur est dirigé vers une page où il peut remplir un formulaire pour modifier les attributs de l'entité produit. L'entité produit comprend des attributs tels que le nom du produit, le prix unitaire, la destination et la catégorie. Ces données sont toutes stockées dans la base de données via Doctrine ORM et peuvent être facilement modifiées par l'administrateur.

Lors de la soumission du formulaire, le contrôleur capture les données du formulaire, persiste les modifications dans l'objet produit correspondant et synchronise ces changements avec la base de données. Tout cela est possible grâce à la méthode edit du contrôleur ProductController, qui utilise le repository ProductRepository pour retrouver le produit à modifier, et le formulaire ProductFormType pour capturer les nouvelles valeurs des attributs du produit.

En termes de sécurité, le formulaire est construit de telle sorte qu'il n'accepte que les données valides. Cela signifie que toutes les données soumises sont vérifiées pour s'assurer qu'elles correspondent aux types de données attendus avant d'être persistées dans la base de données.

Conclusion

